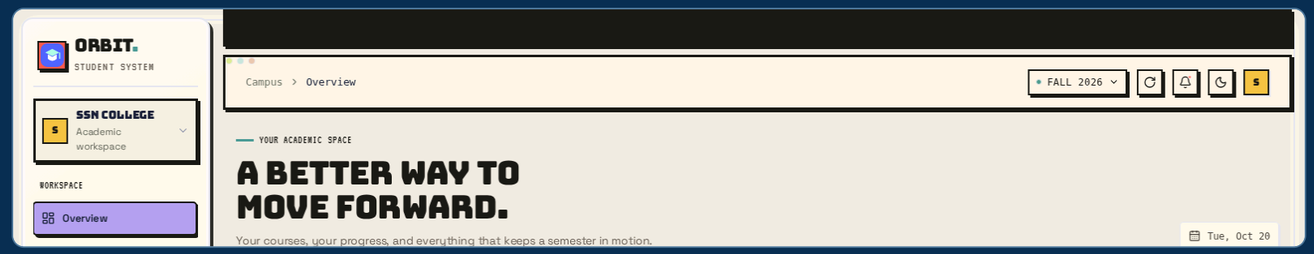
SYSTEM DESIGN LAB · ENGINEERING CASE STUDY · OCTOBER 2026

SCALABLE STUDENT INFORMATION SYSTEM

Correctness first. Performance measured.

COURSE DISCOVERY / ENROLLMENT / ACADEMIC RECORDS



REAL APPLICATION SCREENSHOT · SYNTHETIC DEMO DATA

23/23 Database test suite
23 passed · PostgreSQL-backed

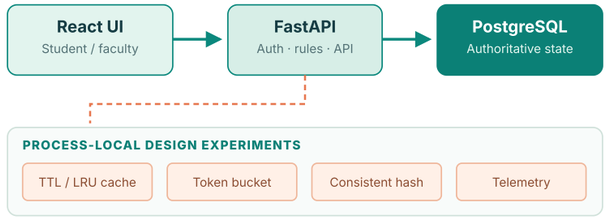
24/24 API checks passed
24 assertions · 13 requests

185 Baseline samples
JMeter · 20 s paced workload

10 ms p95 latency baseline
4.95 ms mean · 9.84 req/s

83.32% Non-success samples under overload
40,372 HTTP 429 · 6 timeouts

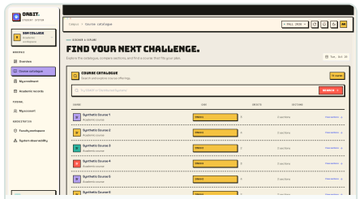
01 / ARCHITECTURE — One authoritative write path



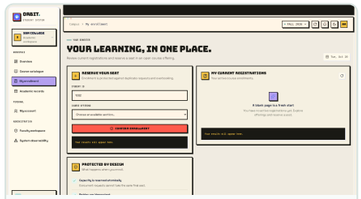
Data flow: browser request → API validation/authentication → domain rule → SQL transaction → response. Course catalogue, offerings, enrollments and academic records use PostgreSQL as the source of truth.

Engineering boundary: cache, rate limiter, hash-ring and telemetry experiments are process-local. The prototype does not claim distributed coordination or PostgreSQL sharding.

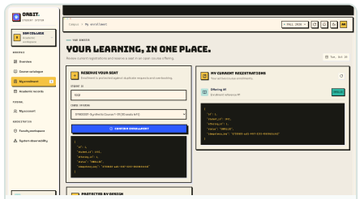
02 / PRODUCT TOUR — Actual interface, annotated by workflow



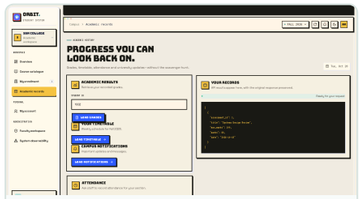
01 · Course discovery
Catalogue and offerings populated from synthetic PostgreSQL data.



02 · Enrollment form
Student selects an offering before submission.



03 · Confirmed registration
Success response and active enrollment record.



04 · Academic records
Assessment results returned by the API.

Captures taken from the actual React production build; all displayed records are synthetic demonstration data.

03 / CONCURRENCY — Twenty students. Five seats.

PostgreSQL-backed concurrent enrollment test · 20 simultaneous students compete for capacity 5.

01 · VALIDATE
Check student, offering and eligibility.

02 · RESERVE
Conditional capacity update in transaction.

03 · INSERT
Create enrollment under unique constraints.

04 · COMMIT
Commit success; rollback conflict.

5 / 20
Accepted · HTTP 201

15 / 20
Rejected · HTTP 409

Observed invariant: enrolled count stays at 5; five active rows, no overbooking. A same-student race also leaves one active enrollment.

Atomic conditional update · **Unique enrollment key** · **Commit / rollback** · **Idempotent retry path**

04 / SYSTEM DESIGN COMPONENTS — Concepts made observable

Rate limiter	Token-bucket admission control; overload requests can return HTTP 429.
Sharding simulation	Routes synthetic key/value records to shard buckets; it does not shard PostgreSQL.
Consistent hashing	Ring-based key routing and node-change movement experiment; no automatic data migration.
Key-value cache	TTL/LRU cache for autocomplete, with observable MISS/HIT behavior.
Unique IDs	Snowflake-style 64-bit IDs; API serializes IDs as strings to avoid JavaScript precision loss.
Short URL	Create and redirect through the application's persistent short-link path.
Bounded crawler	URL frontier and safety limits are demonstrable; live public crawling remains unverified.
Autocomplete	Prefix-based course suggestions with cache-backed repeated lookup.

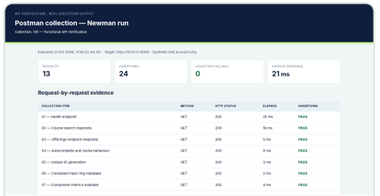
05 / VERIFICATION — Measured, not assumed

23 / 23 passed
Automated tests against PostgreSQL

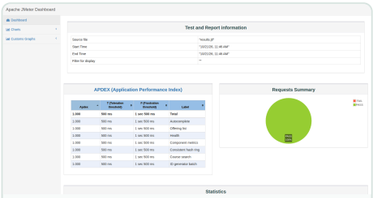
24 / 24 passed
Postman/Newman assertions · 13 requests

185 samples
JMeter paced baseline · 10 threads · 20 s

0 errors
Baseline mean 4.95 ms · p95 10 ms



API contract checks
Health, catalogue, IDs, auth and role-denial assertions.



Paced load baseline
9.84 requests/s in the fresh local rerun.

Separate overload experiment: 40,372 of 48,459 unpaced samples returned HTTP 429 (83.32%); 8,081 returned HTTP 200 and 6 timed out. Failed samples include transport timeouts, so this is admission-control evidence—not baseline capacity.

Local single-laptop measurements; not production capacity certification. A second paced run had p95 20 ms, so repeat runs are needed for stronger claims.

06 / DESIGN BOUNDARIES — What is—and isn't—scaled

Integrity
Capacity guarded inside the database transaction.

Measured workload
Paced baseline and unpaced overload are reported separately.

Process-local state
Cache, limiter and hash ring do not coordinate across workers.

Safe IDs
64-bit IDs are serialized as strings for JavaScript clients.

Not yet proven
Multi-node scaling and production capacity are not claimed.

Next validation
Repeat baselines and sample API, database and load-generator resources.

TEAM
Tanish S K (3122245001180) · Tarun S (3122245001182) · Sujith M (3122245001172) · Velamapalli Hemanth Raj Sreyash (3122245001195)
System Design Lab · Scalable Student Information System